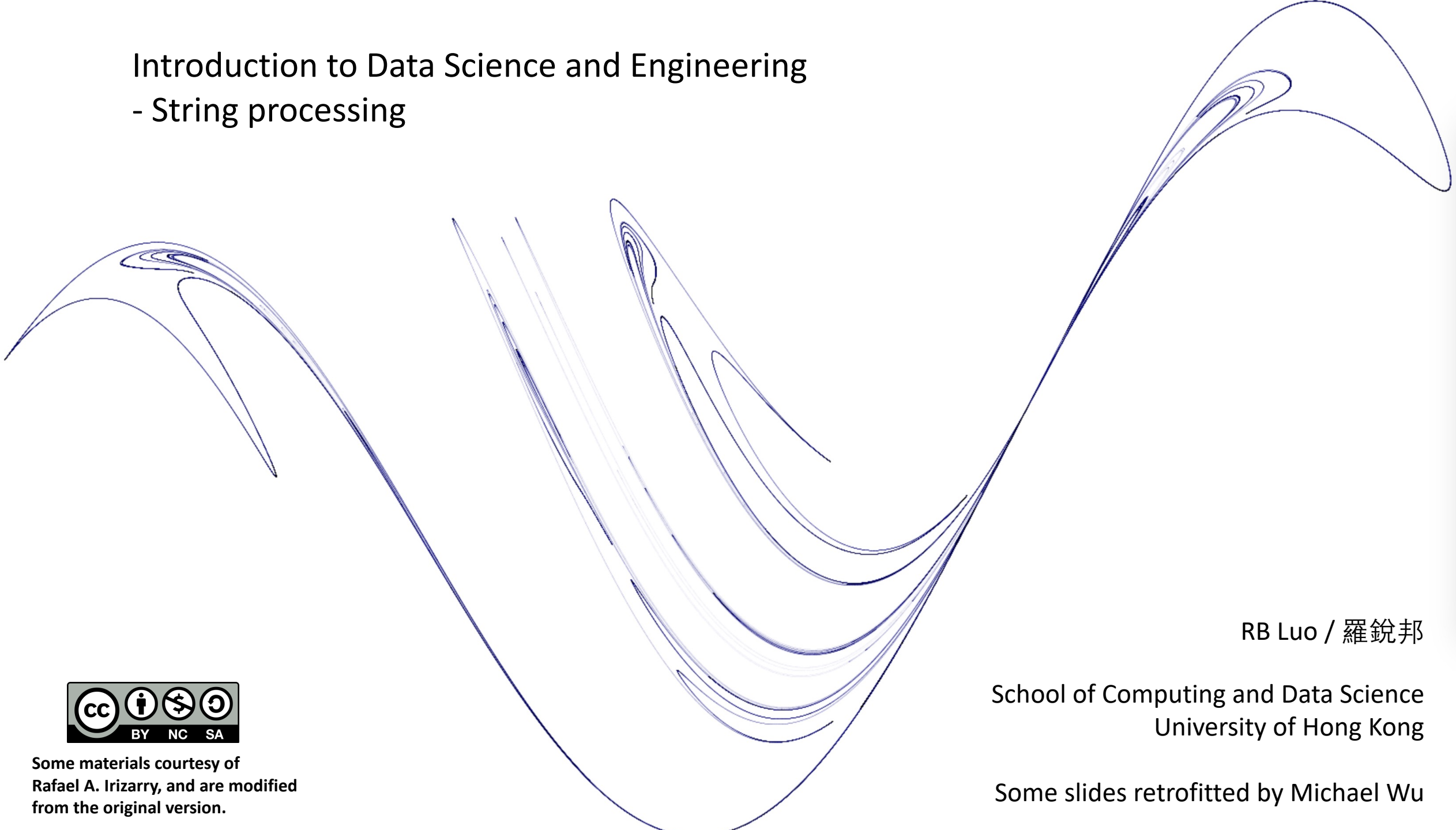
# Introduction to Data Science and Engineering
# - String processing



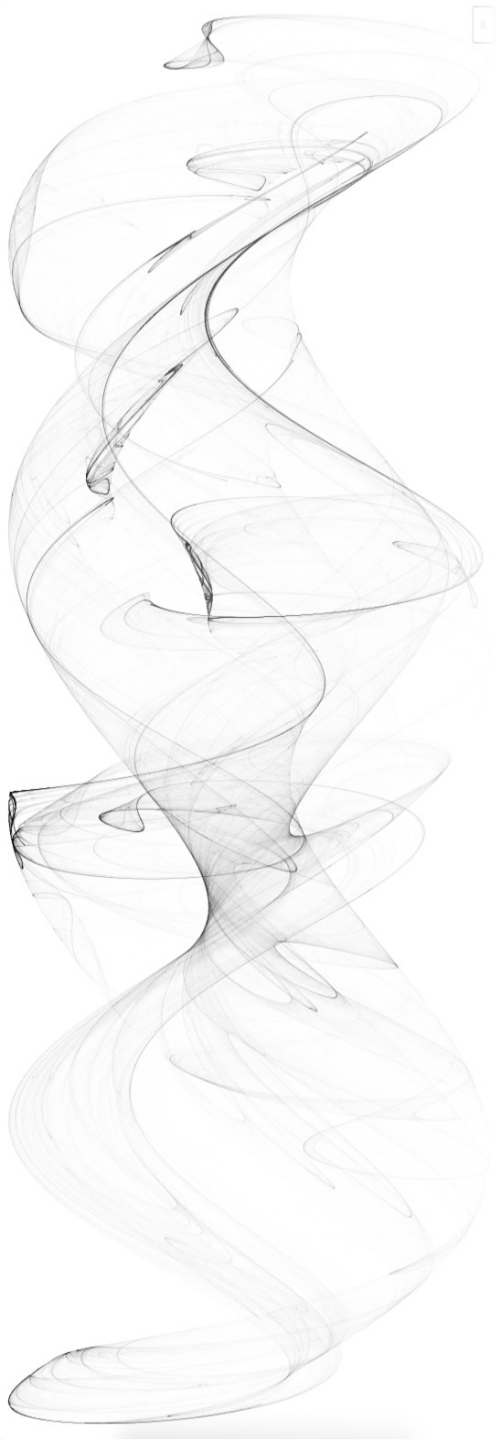
RB Luo / 羅銳邦

School of Computing and Data Science
University of Hong Kong

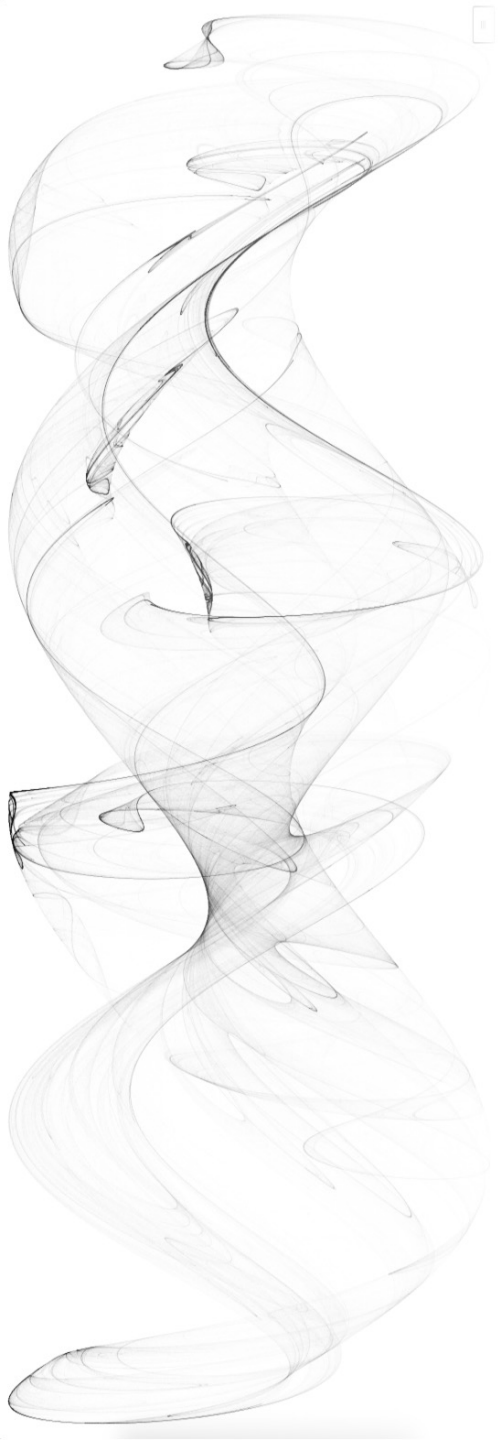
Some slides retrofitted by Michael Wu



**Some materials courtesy of Rafael A. Irizarry, and are modified from the original version.**



- One of the most common data wrangling challenges involves extracting numeric data contained in character strings and converting them into the numeric representations required to make plots, compute summaries, or fit models in R
- Also common is processing unorganized text into meaningful variable names or categorical variables
- String processing seems to be simple, but is usually one of the most annoying tasks
- Using examples of converting raw data to datasets including murders, heights, and research_funding_rates, we will show the most common tasks in string processing including extracting numbers from strings, removing unwanted characters from text, finding and replacing characters, extracting specific parts of strings, converting free form text to more uniform formats, and splitting strings into multiple values
- We will use the stringr package
- We will learn the basics of regex (regular expression)



# stringr

- Part of `tidyverse`;
- All `stringr` functions take a character vector as first argument, thus they can be used with piping;
- All `stringr` operations are vectorized.

# Work with strings with stringr : : **CHEAT SHEET**

The **stringr** package provides a set of internally consistent tools for working with character strings, i.e. sequences of characters surrounded by quotation marks.

## Detect Matches


**str_detect**(string, **pattern**) Detect the presence of a pattern match in a string.
*str_detect(fruit, "a")*


**str_which**(string, **pattern**) Find the indexes of strings that contain a pattern match.
*str_which(fruit, "a")*


**str_count**(string, **pattern**) Count the number of matches in a string.
*str_count(fruit, "a")*


**str_locate**(string, **pattern**) Locate the positions of pattern matches in a string. Also **str_locate_all**. *str_locate(fruit, "a")*

## Subset Strings

**str_sub**(string, start = 1L, end = -1L) Extract substrings from a character vector.
*str_sub(fruit, 1, 3); str_sub(fruit, -2)*

**str_subset**(string, **pattern**) Return only the strings that contain a pattern match.
*str_subset(fruit, "b")*


**str_extract**(string, **pattern**) Return the first pattern match found in each string, as a vector. Also **str_extract_all** to return every pattern match. *str_extract(fruit, "[aeiou]")*


**str_match**(string, **pattern**) Return the first pattern match found in each string, as a matrix with a column for each ( ) group in pattern. Also **str_match_all**.
*str_match(sentences, "(a|the) ([^ ]+)")*

## Manage Lengths


**str_length**(string) The width of strings (i.e. number of code points, which generally equals the number of characters). *str_length(fruit)*

**str_pad**(string, width, side = c("left", "right", "both"), pad = " ") Pad strings to constant width. *str_pad(fruit, 17)*

**str_trunc**(string, width, side = c("right", "left", "center"), ellipsis = "...") Truncate the width of strings, replacing content with ellipsis.
*str_trunc(fruit, 3)*

**str_trim**(string, side = c("both", "left", "right")) Trim whitespace from the start and/or end of a string. *str_trim(fruit)*

## Mutate Strings

**str_sub**() **<-** value. Replace substrings by identifying the substrings with str_sub() and assigning into the results.
*str_sub(fruit, 1, 3) <- "str"*

**str_replace**(string, **pattern**, replacement) Replace the first matched pattern in each string. *str_replace(fruit, "a", "-")*

**str_replace_all**(string, **pattern**, replacement) Replace all matched patterns in each string. *str_replace_all(fruit, "a", "-")*

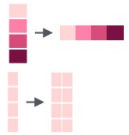

**str_to_lower**(string, locale = "en")[1] Convert strings to lower case.
*str_to_lower(sentences)*


**str_to_upper**(string, locale = "en")[1] Convert strings to upper case.
*str_to_upper(sentences)*


**str_to_title**(string, locale = "en")[1] Convert strings to title case. *str_to_title(sentences)*

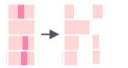
## Join and Split

**str_c**(..., sep = "", collapse = NULL) Join multiple strings into a single string.
*str_c(letters, LETTERS)*

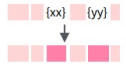


**str_c**(..., sep = "", **collapse = NULL**) Collapse a vector of strings into a single string.
*str_c(letters, collapse = "")*

**str_dup**(string, times) Repeat strings times times. *str_dup(fruit, times = 2)*



**str_split_fixed**(string, **pattern**, n) Split a vector of strings into a matrix of substrings (splitting at occurrences of a pattern match). Also **str_split** to return a list of substrings.
*str_split_fixed(fruit, " ", n=2)*




glue::**glue**(..., .sep = "", .envir = parent.frame(), .open = "{", .close = "}") Create a string from strings and {expressions} to evaluate. *glue::glue("Pi is {pi}")*

glue::**glue_data**(.x, ..., .sep = "", .envir = parent.frame(), .open = "{", .close = "}") Use a data frame, list, or environment to create a string from strings and {expressions} to evaluate. *glue::glue_data(mtcars, "{rownames(mtcars)} has {hp} hp")*

## Order Strings


**str_order**(x, decreasing = FALSE, na_last = TRUE, locale = "en", numeric = FALSE, ...)[1] Return the vector of indexes that sorts a character vector. *x[str_order(x)]*

**str_sort**(x, decreasing = FALSE, na_last = TRUE, locale = "en", numeric = FALSE, ...)[1] Sort a character vector.
*str_sort(x)*

## Helpers

```
apple
banana
pear
```

```
apple
banana
pear
```

**str_conv**(string, encoding) Override the encoding of a string. *str_conv(fruit,"ISO-8859-1")*

**str_view**(string, **pattern**, match = NA) View HTML rendering of first regex match in each string. *str_view(fruit, "[aeiou]")*

**str_view_all**(string, **pattern**, match = NA) View HTML rendering of all regex matches.
*str_view_all(fruit, "[aeiou]")*

**str_wrap**(string, width = 80, indent = 0, exdent = 0) Wrap strings into nicely formatted paragraphs. *str_wrap(sentences, 20)*

See also: https://rstudio.github.io/cheatsheets/html/strings.html
https://github.com/rstudio/cheatsheets/blob/main/strings.pdf

# Case study 1: US murders data

This is where we ended after learning web scraping

```
R 4.2.2 · ~/
> library(rvest)
> url <- paste0("https://en.wikipedia.org/w/index.php?title=",
+               "Gun_violence_in_the_United_States_by_state",
+               "&direction=prev&oldid=810166167")
> murders_raw <- read_html(url) |>
+ html_node("table") |>
+ html_table() |>
+ setNames(c("state", "population", "total", "murder_rate"))
> murders_raw
# A tibble: 51 × 4
   state                population total murder_rate
   <chr>                <chr>      <chr>        <dbl>
 1 Alabama              4,853,875  348            7.2
 2 Alaska               737,709    59             8
 3 Arizona              6,817,565  309            4.5
 4 Arkansas             2,977,853  181            6.1
 5 California           38,993,940 1,861          4.8
 6 Colorado             5,448,819  176            3.2
 7 Connecticut          3,584,730  117            3.3
 8 Delaware             944,076    63             6.7
 9 District of Columbia 670,377    162           24.2
10 Florida              20,244,914 1,041          5.1
# … with 41 more rows
# ℹ Use `print(n = ...)` to see more rows
```

Just in case you need it: https://en.wikipedia.org/w/index.php?title=Gun_violence_in_the_United_States_by_state&direction=prev&oldid=810166167

as.numeric() doesn't work due to thousand comma separators

```
R 4.2.2 · ~/
> as.numeric(murders_raw$population)
 [1] NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
[23] NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA NA
[45] NA NA NA NA NA NA NA
Warning message:
NAs introduced by coercion
```

To detect which columns contain ",", we use str_detect

```
R 4.2.2 · ~/
> str_detect(murders_raw$state, ",") |> any()
[1] FALSE
> str_detect(murders_raw$population, ",") |> any()
[1] TRUE
> str_detect(murders_raw$total, ",") |> any()
[1] TRUE
> str_detect(murders_raw$murder_rate, ",") |> any()
[1] FALSE
```

# str_replace_all

- When processing strings
  - Test run before changing the original table
  - Work on just a column at a time

```
R 4.2.2 · ~/
> test_1 <- str_replace_all(murders_raw$population, ",", "")
> test_1 <- as.numeric(test_1)
> test_1
 [1]  4853875   737709  6817565  2977853 38993940  5448819  3584730
 [8]   944076   670377 20244914 10199398  1425157  1652828 12839047
[15]  6612768  3121997  2906721  4424611  4668960  1329453  5994983
[22]  6784240  9917715  5482435  2989390  6076204  1032073  1893765
[29]  2883758  1330111  8935421  2080328 19747183 10035186   756835
[36] 11605090  3907414  4024634 12791904  1055607  4894834   857919
[43]  6595056 27429639  2990632   626088  8367587  7160290  1841053
[50]  5767891   586107
> murders_raw$population
 [1] "4,853,875"  "737,709"    "6,817,565"  "2,977,853"  "38,993,940"
 [6] "5,448,819"  "3,584,730"  "944,076"    "670,377"    "20,244,914"
[11] "10,199,398" "1,425,157"  "1,652,828"  "12,839,047" "6,612,768"
[16] "3,121,997"  "2,906,721"  "4,424,611"  "4,668,960"  "1,329,453"
[21] "5,994,983"  "6,784,240"  "9,917,715"  "5,482,435"  "2,989,390"
[26] "6,076,204"  "1,032,073"  "1,893,765"  "2,883,758"  "1,330,111"
[31] "8,935,421"  "2,080,328"  "19,747,183" "10,035,186" "756,835"
[36] "11,605,090" "3,907,414"  "4,024,634"  "12,791,904" "1,055,607"
[41] "4,894,834"  "857,919"    "6,595,056"  "27,429,639" "2,990,632"
[46] "626,088"    "8,367,587"  "7,160,290"  "1,841,053"  "5,767,891"
[51] "586,107"
```

Cross-check

Then mutate:

```
R 4.2.2 · ~/
> murders_processed_1 =
+ mutate(murders_raw,
+        population = as.numeric(str_replace_all(population, ",", "")))
> murders_processed_1
# A tibble: 51 × 4
   state                population total murder_rate
   <chr>                     <dbl> <chr>       <dbl>
 1 Alabama                 4853875 348           7.2
 2 Alaska                   737709 59            8
 3 Arizona                 6817565 309           4.5
 4 Arkansas                2977853 181           6.1
 5 California             38993940 1,861         4.8
 6 Colorado                5448819 176           3.2
 7 Connecticut             3584730 117           3.3
 8 Delaware                 944076 63            6.7
 9 District of Columbia     670377 162          24.2
10 Florida                20244914 1,041         5.1
# … with 41 more rows
# ℹ Use `print(n = ...)` to see more rows
```

# parse_number

Removing thousand comma separators is too common an operation, we can use parse_number instead

R 4.2.2 · ~/

```
> test_2 <- parse_number(murders_raw$population)
> identical(test_1, test_2)
[1] TRUE
> test_2
 [1]  4853875   737709  6817565  2977853 38993940  5448819  3584730
 [8]   944076   670377 20244914 10199398  1425157  1652828 12839047
[15]  6612768  3121997  2906721  4424611  4668960  1329453  5994983
[22]  6784240  9917715  5482435  2989390  6076204  1032073  1893765
[29]  2883758  1330111  8935421  2080328 19747183 10035186   756835
[36] 11605090  3907414  4024634 12791904  1055607  4894834   857919
[43]  6595056 27429639  2990632   626088  8367587  7160290  1841053
[50]  5767891   586107
```

# Case study 2: self-reported heights

The dataset we used loaded from `data(heights)` was processed. Let's try data wrangling from the raw data

R 4.2.2 · ~/

```
> data(reported_heights)
> reported_heights
             time_stamp    sex                height
1   2014-09-02 13:40:36   Male                    75
2   2014-09-02 13:46:59   Male                    70
3   2014-09-02 13:59:20   Male                    68
4   2014-09-02 14:51:53   Male                    74
5   2014-09-02 15:16:15   Male                    61
6   2014-09-02 15:16:16 Female                    65
7   2014-09-02 15:16:19 Female                    66
8   2014-09-02 15:16:21 Female                    62
9   2014-09-02 15:16:21 Female                    66
10  2014-09-02 15:16:22   Male                    67
11  2014-09-02 15:16:22   Male                    72
12  2014-09-02 15:16:23   Male                     6
13  2014-09-02 15:16:23   Male                    69
14  2014-09-02 15:16:26   Male                    68
15  2014-09-02 15:16:26   Male                    69
16  2014-09-02 15:16:26   Male                    66
17  2014-09-02 15:16:27   Male                    75
18  2014-09-02 15:16:27 Female                    64
19  2014-09-02 15:16:27 Female                    60
20  2014-09-02 15:16:28   Male                    67
21  2014-09-02 15:16:28   Male                    66
22  2014-09-02 15:16:28   Male                  5' 4"
```

These heights were obtained using a google survey in which students were asked to enter their heights. They could enter anything, but the instructions asked for height in inches, a number. We compiled 1,095 submissions, but unfortunately the column vector with the reported heights had several non-numeric entries and as a result became a character vector

R 4.2.2 · ~/

```
> class(reported_heights$height)
[1] "character"
```

If we try to convert it into numbers, we get a warning

```
R 4.2.2 · ~/
> x <- as.numeric(reported_heights$height)
Warning message:
NAs introduced by coercion
```

81 rows end up not convertible to number

```
R 4.2.2 · ~/
> sum(is.na(x))
[1] 81
```

We can see some of the entries that are not successfully converted by using filter to keep only the entries resulting in NAs

```
R 4.2.2 · ~/
> reported_heights |>
+ mutate(new_height = as.numeric(height)) |>
+ filter(is.na(new_height)) |>
+ head(10)
           time_stamp    sex                  height new_height
1  2014-09-02 15:16:28   Male                   5' 4"         NA
2  2014-09-02 15:16:37 Female                   165cm         NA
3  2014-09-02 15:16:52   Male                     5'7         NA
4  2014-09-02 15:16:56   Male                   >9000         NA
5  2014-09-02 15:16:56   Male                    5'7"         NA
6  2014-09-02 15:17:09 Female                    5'3"         NA
7  2014-09-02 15:18:00   Male 5 feet and 8.11 inches         NA
8  2014-09-02 15:19:48   Male                    5'11         NA
9  2014-09-04 00:46:45   Male                   5'9''         NA
10 2014-09-04 10:29:44   Male                  5'10''         NA
```

We immediately found what's happening, many reporting in feet and inches, not just inches as requested

For those convertible to numbers, are they all reasonable? Let's assume any converted numbers above 84 inches or below 50 are susceptible

R 4.2.2 · ~/

```
> reported_heights |>
+ mutate(new_height = as.numeric(height)) |>
+ filter(new_height > 84 | new_height < 50)
```

| | time_stamp | sex | height | new_height |
|---|---|---|---|---|
| 1 | 2014-09-02 15:16:23 | Male | 6 | 6.00 |
| 2 | 2014-09-02 15:16:32 | Female | 5.3 | 5.30 |
| 3 | 2014-09-02 15:16:41 | Male | 511 | 511.00 |
| 4 | 2014-09-02 15:16:46 | Male | 6 | 6.00 |
| 5 | 2014-09-02 15:16:50 | Female | 2 | 2.00 |
| 6 | 2014-09-02 15:18:14 | Female | 5.25 | 5.25 |
| 7 | 2014-09-03 21:43:00 | Male | 5.5 | 5.50 |
| 8 | 2014-09-03 23:55:37 | Male | 11111 | 11111.00 |
| 9 | 2014-09-04 05:15:28 | Female | 6 | 6.00 |
| 10 | 2014-09-04 06:31:03 | Male | 6.5 | 6.50 |
| 11 | 2014-09-04 09:24:41 | Female | 150 | 150.00 |
| 12 | 2014-09-04 14:23:19 | Male | 103.2 | 103.20 |
| 13 | 2014-09-06 17:01:32 | Male | 5.8 | 5.80 |
| 14 | 2014-09-06 21:32:15 | Male | 19 | 19.00 |
| 15 | 2014-09-06 22:38:40 | Female | 5 | 5.00 |
| 16 | 2014-09-07 14:35:35 | Female | 5.6 | 5.60 |
| 17 | 2014-09-07 16:33:46 | Male | 175 | 175.00 |
| 18 | 2014-09-07 18:18:31 | Male | 177 | 177.00 |

R 4.2.2 · ~/

```
> reported_heights |>
+ mutate(new_height = as.numeric(height)) |>
+ filter(new_height > 84 | new_height < 50) |> dim()
[1] 211   4
```

There are 211 rows

These sanity checks are better put together in a function so to be used again and again during string processing

```
R 4.2.2 · ~/
> not_inches <- function(x, smallest = 50, tallest = 84){
+     inches <- suppressWarnings(as.numeric(x))
+     ind <- is.na(inches) | inches < smallest | inches > tallest
+     ind
+ }
```

We apply this function and find the problematic entries, there are 292 of them

```
R 4.2.2 · ~/
> problems <- reported_heights |>
+ filter(not_inches(height)) |>
+ pull(height)
> length(problems)
[1] 292
```

```
R 4.2.2 · ~/
> paste(problems, collapse="  ")
[1] "6  5' 4\"  5.3  165cm  511  6  2  5'7  >9000  5'7\"  5'3\"  5 feet and
8.11 inches  5.25  5'11  5.5  11111  5'9''  6  6.5  150  5'10''  103.2  5.8
19  5  5.6  175  177  300  5,3  6'  6  5.9  6,8  5' 10  5.5  178  163  6.2  1
75  Five foot eight inches  6.2  5.8  5.1  178  165  5.11  5'5\"  165  180
5'2\"  5.75  169  5,4  7  5.4  157  6.1  169  5'3  5.6  214  183  5.6  6  162
178  180  5'10''  170  5'3''  178  0.7  190  5.4  184  5'7''  5.9  5'12  5.6
5.6  184  6  167  2'33  5'11  5'3\"  5.5  5.2  180  5.5  5.5  6.5  5,8  180
183  170  5'6''  172  612  5.11  168  5'4  1,70  172  87  5.5  176  5'7.5''
5'7.5''  111  5'2\"  173  174  176  175  5' 7.78\"  6.7  12  6  5.1  5.6  5.5
yyy  5.2  5'5  5'8  5'6  5 feet 7inches  89  5.6  5.7  183  172  34  25  6
5.9  168  6.5  170  175  6  22  5.11  684  6  1  1  6*12  5 .11  87  162  165
184  6  173  1.6  172  170  5.7  5.5  174  170  160  120  120  23  192  5 11
167  150  1.7  174  5.8  6  5'4  5'8\"  5'5  5.8  5.1  5.11  5.7  5'7  5'6
5'11\"  5'7\"  5'7  172  5'8  180  5' 11\"  5  180  180  6'1\"  5.9  5.2  5.5
69\"  5' 7\"  5'10''  5.51  5'10  5'10  5ft 9 inches  5 ft 9 inches  5'2  5'1
1  5.8  5.7  167  168  6  6.1  5'11''  5.69  178  182  164  5'8\"  185  6  86
5.7  708,661  5.25  5.5  5 feet 6 inches  5'10''  172  6  5'8  160  6'3\"  64
9,606  10000  5.1  152  1  180  728,346  175  158  173  164  6 04  169  0  18
5  168  5'9  169  5'5''  174  6.3  179  5'7\"  5.5  6  6  170  6  172  158  1
00  159  190  5.7  170  158  6'4\"  180  5.57  5'4  210  88  6  162  170 cm
5.7  170  157  186  170  7,283,465  5  5  34  161  5'6  5'6"
```

Show them all (or enough entries). Look into them and identify common mistake patterns. Try finding the fewest steps of data wrangling and string processing to either correct or remove the mistakes

```
R 4.2.2 · ~/
> paste(problems, collapse="  ")
[1] "6  5' 4\"  5.3  165cm  511  6  2  5'7  >9000  5'7\"  5'3\"  5 feet and
8.11 inches  5.25  5'11  5.5  11111  5'9''  6  6.5  150  5'10''  103.2  5.8
19  5  5.6  175  177  300  5,3  6'  6  5.9  6,8  5' 10  5.5  178  163  6.2  1
75  Five foot eight inches  6.2  5.8  5.1  178  165  5.11  5'5\"  165  180
5'2\"  5.75  169  5,4  7  5.4  157  6.1  169  5'3  5.6  214  183  5.6  6  162
178  180  5'10''  170  5'3''  178  0.7  190  5.4  184  5'7''  5.9  5'12  5.6
5.6  184  6  167  2'33  5'11  5'3\"  5.5  5.2  180  5.5  5.5  6.5  5,8  180
183  170  5'6''  172  612  5.11  168  5'4  1,70  172  87  5.5  176  5'7.5''
5'7.5''  111  5'2\"  173  174  176  175  5' 7.78\"  6.7  12  6  5.1  5.6  5.5
yyy  5.2  5'5  5'8  5'6  5 feet 7inches  89  5.6  5.7  183  172  34  25  6
5.9  168  6.5  170  175  6  22  5.11  684  6  1  1  6*12  5 .11  87  162  165
184  6  173  1.6  172  170  5.7  5.5  174  170  160  120  120  23  192  5 11
167  150  1.7  174  5.8  6  5'4  5'8\"  5'5  5.8  5.1  5.11  5.7  5'7  5'6
5'11\"  5'7\"  5'7  172  5'8  180  5' 11\"  5  180  180  6'1\"  5.9  5.2  5.5
69\"  5' 7\"  5'10''  5.51  5'10  5'10  5ft 9 inches  5 ft 9 inches  5'2  5'1
1  5.8  5.7  167  168  6  6.1  5'11''  5.69  178  182  164  5'8\"  185  6  86
5.7  708,661  5.25  5.5  5 feet 6 inches  5'10''  172  6  5'8  160  6'3\"  64
9,606  10000  5.1  152  1  180  728,346  175  158  173  164  6 04  169  0  18
5  168  5'9  169  5'5''  174  6.3  179  5'7\"  5.5  6  6  170  6  172  158  1
00  159  190  5.7  170  158  6'4\"  180  5.57  5'4  210  88  6  162  170 cm
5.7  170  157  186  170  7,283,465  5  5  34  161  5'6  5'6"
```

There are three common mistake patterns:
1. A pattern of the form x'y or x' y'' or x'y" with x and y respectively representing feet and inches
2. A pattern of the form x.y or x,y with x feet and y inches
3. Entries that were reported in centimeters rather than inches.

There are some entries written in natural language such as “Five foot eight inches”, “5 feet 7inches”. Let’s consider them later

There are some other irrational entries such as “>9000”, “11111”, “708,661”. Consider them as nonstarters to be removed at the end. We just need to ensure that there are only a small number of them.

According to the three common mistake patterns, we can develop a plan of attack

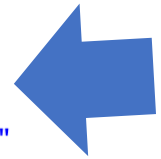
**Plan of attack**: we will convert entries fitting the first two patterns into a standardized one. We will then leverage the standardization to extract the feet and inches and convert to inches. We will then define a procedure for identifying entries that are in centimeters and convert them to inches. After applying these steps, we will then check again to see what entries were not fixed and see if we can tweak our approach to be more comprehensive.

At the end, we hope to have a script that makes web-based data collection methods robust to the most common user mistakes.

To achieve our goal, we will use a technique that enables us to accurately detect patterns and extract the parts we want: regular expressions (regex).

Remember that there is rarely just one way to perform these tasks. Here we pick one that helps us teach several useful techniques. But surely there is a more efficient way of performing the task

```
R 4.2.2 · ~/
> hello_str <- "Hello!"
> hello_str <- 'Hello!'
> hello_str <- `Hello!`
Error: object 'Hello!' not found
> complex_"var"_name <- hello_str
Error: unexpected string constant in "complex_"var""
> `complex_"var"_name` <- hello_str
> print(complex_"var"_name)
Error: unexpected string constant in "print(complex_"var""
> print(`complex_"var"_name`)
[1] "Hello!"
> `complex_"var"_name` <- "i am 6'0""
+
+
> `complex_"var"_name` <- "i am 6'0\""
> `complex_"var"_name` <- 'i am 6'0"'
Error: unexpected numeric constant in "`complex_"var"_name` <- 'i am 6'0"
> `complex_"var"_name` <- 'i am 6\'0"'
> `complex_"var"_name` <- 'i am 72"'
> `complex_"var"_name` <- "i am 6'0"
```



Will stuck here, ctrl-c

First, we quickly describe how to escape the function of certain characters so that they can be included in strings

To define strings in R, we can use either double quotes or single quotes

Backticks is used when you have special characters in the variable name. Technically, in R, code in backticks is not evaluated and used as is

To use a double quote in double quotes:

- Use backslash as an escape
- Mix use of ‘ ’ and “ ”

My suggestion is to always use backslash, there are cases that mix use is not enough, like our case

# Regular expressions

A regular expression (regex) is a way to describe specific patterns of characters of text. They can be used to determine if a given string matches the pattern. A set of rules has been defined to do this efficiently and precisely

The patterns supplied to the stringr functions can be a regex rather than a standard string. We will learn how this works through a series of examples. We will create strings to test out our regex. We define patterns that should match and those should not. We will call them yes and no, respectively. This permits us to check for the two types of errors: failing to match (aka false negative, type 2 error) and incorrectly matching (aka false positive, type 1 error)

Online tutorial:
https://www.regular-expressions.info/tutorial.html
https://r4ds.had.co.nz/strings.html#matching-patterns-with-regular-expressions
Cheat sheet: https://github.com/rstudio/cheatsheets/blob/main/regex.pdf

# Strings are a regexp

```
R 4.2.2 · ~/
> pattern <- ","
> mutate(murders_raw, contain_comma = str_detect(total, pattern)) |>
+ select(total, contain_comma)
# A tibble: 51 × 2
   total contain_comma
   <chr> <lgl>
 1 348   FALSE
 2 59    FALSE
 3 309   FALSE
 4 181   FALSE
 5 1,861 TRUE
 6 176   FALSE
 7 117   FALSE
 8 63    FALSE
 9 162   FALSE
10 1,041 TRUE
# … with 41 more rows
# ℹ Use `print(n = ...)` to see more rows
```

Technically any string is a regex, perhaps the simplest example is a single character. So, the comma used in the next code example is a simple example of searching with regex

```
R 4.2.2 · ~/
> str_subset(reported_heights$height, "cm")
[1] "165cm"  "170 cm"
```

In reported_heights, we noted some entries included “cm”. This is also a simple example of a regex. We can show all the entries that used cm like this

# Special character

Let’s consider a slightly more complicated example. Which of the following strings contain the pattern cm or inches?

```
R 4.2.2 · ~/
> yes <- c("180 cm", "70 inches")
> no <- c("180", "70''")
> s <- c(yes, no)
> str_detect(s, "cm") | str_detect(s, "inches")
[1]  TRUE  TRUE FALSE FALSE
```

However, we don’t need to do this. The main feature that distinguishes the regex language from plain strings is that we can use special characters. These are characters with a meaning. We start by introducing | which means or.

```
R 4.2.2 · ~/
> str_detect(s, "cm|inches")
[1]  TRUE  TRUE FALSE FALSE
```

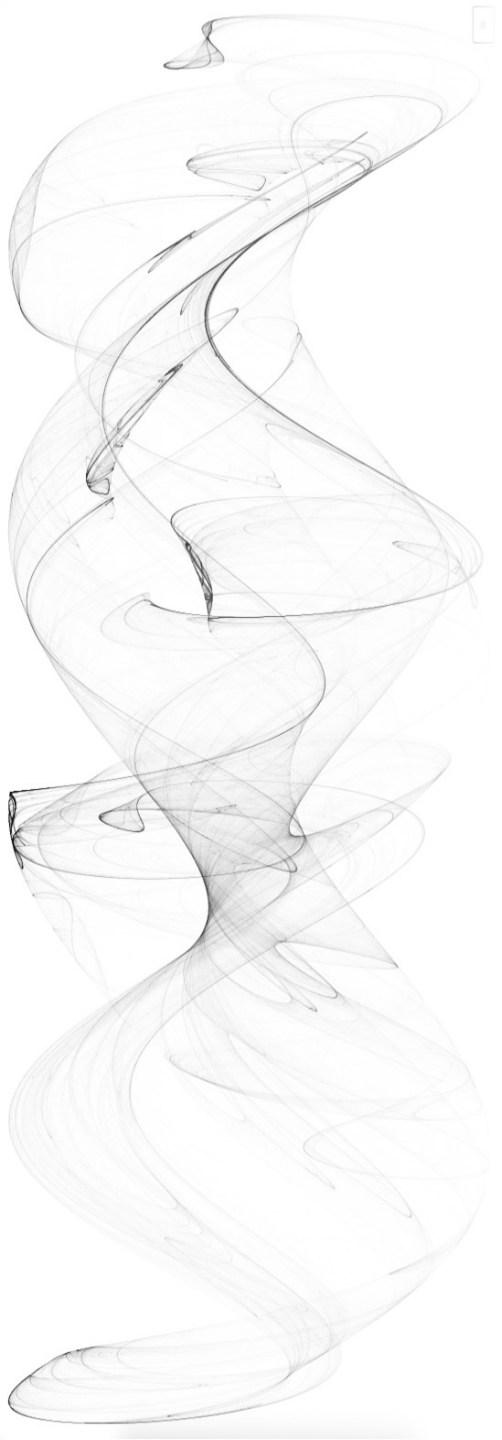
# Special character

```
R 4.2.2 · ~/
> yes <- c("5", "6", "5'10", "5 feet", "4'11")
> no <- c("", ".", "Five", "six")
> s <- c(yes, no)
> pattern <- "\\d"
> str_detect(s, pattern)
[1]  TRUE  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE
```

Another special character that will be useful for identifying feet and inches values is \d which means any digit: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9. The backslash is used to distinguish it from the character d. In R, we have to escape the backslash \ so we actually need to use \\d to represent digits.

```
R 4.2.2 · ~/
> str_view_all(s, pattern)
[1] | <5>
[2] | <6>
[3] | <5>'<1><0>
[4] | <5> feet
[5] | <4>'<1><1>
[6] |
[7] | .
[8] | Five
[9] | six
```

We take this opportunity to introduce the str_view_all function, which is helpful for troubleshooting as it shows us the matches for each string



# Regular expressions

There are many other special characters. You can see most of them in the cheat sheet, but here I introduce a few more that are commonly used

\\D stands for everything this is not \\d

\\w stands for word character and it matches any letter, number, or underscore. It is equivalent to [a-zA-Z0-9_]

\\W stands for everything that is not \\w

\\s stands for space, tab, vertical tab, newline, form feed, carriage return

\\S stands for everything that is not \\s

We will see more examples later

Online tutorial:
https://www.regular-expressions.info/tutorial.html
https://r4ds.had.co.nz/strings.html#matching-patterns-with-regular-expressions
Cheat sheet: https://github.com/rstudio/cheatsheets/blob/main/regex.pdf

# Character class

```
R 4.2.2 · ~/
> str_view_all(s, "[56]")
[1] | <5>
[2] | <6>
[3] | <5>'10
[4] | <5> feet
[5] | 4'11
[6] |
[7] | .
[8] | Five
[9] | six
```

Character class are used to define a series of characters that can be matched. We define character class with square brackets []. So, for example, if we want the pattern to match only if we have a 5 or a 6, we use the regex [56]

```
R 4.2.2 · ~/
> yes <- as.character(4:7)
> no <- as.character(1:3)
> s <- c(yes, no)
> str_detect(s, "[4-7]")
[1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE
```

Suppose we want to match values between 4 and 7. A common way to define character classes is with ranges. So, for example, [0-9] is equivalent to \\d. The pattern we want is therefore [4-7]

**Caveats: It is important to know that in regex everything is a character**; there are no numbers. So 4 is the character 4 not the number four. Notice, for example, that [1-20] does not mean 1 through 20, it means the characters 1 through 2 or the character 0. **So [1-20] simply means the character class composed of 0, 1, and 2.**

Keep in mind that characters do have an order and the digits do follow the numeric order. So 0 comes before 1 which comes before 2 and so on. For the same reason, we can define lower case letters as [a-z], upper case letters as [A-Z], and [a-zA-z] as both

# Anchor

```
R 4.2.2 · ~/
> pattern <- "^\\d$"
> yes <- c("1", "5", "9")
> no <- c("12", "123", " 1", "a4", "b")
> s <- c(yes, no)
> str_view_all(s, pattern)
[1] | <1>
[2] | <5>
[3] | <9>
[4] | 12
[5] | 123
[6] |  1
[7] | a4
[8] | b
```

What if we want a match when we have exactly 1 digit? This will be useful in our case study since feet are never more than 1 digit so a restriction will help us. **One way to do this** with regex is by using anchor, which let us define patterns that must start or end at a specific place. The two most common anchors are ^ and $ which represent the beginning and end of a string, respectively. So the pattern ^\\d$ is read as “start of the string followed by one digit followed by end of string”

The 1 does not match because it does not start with the digit but rather with a space, which is actually not easy to see

# Quantifier {}

```
R 4.2.2 · ~/
> pattern <- "^\\d{1,2}$"
> yes <- c("1", "5", "9", "12")
> no <- c("123", "a4", "b")
> str_view_all(c(yes, no), pattern)
[1] | <1>
[2] | <5>
[3] | <9>
[4] | <12>
[5] | 123
[6] | a4
[7] | b
```

For the inches part, we can have one or two digits. This can be specified in regex with quantifier. This is done by following the pattern with curly brackets containing the number of times the previous entry can be repeated.

In this case, 123 does not match, but 12 does

# First try to put things together

```
R 4.2.2 · ~/
> pattern <- "^[4-7]'\\d{1,2}\"$"
```

^ = start of the string

[4-7] = one digit, either 4,5,6 or 7

' = feet symbol

\\d{1,2} = one or two digits

\" = inches symbol

$ = end of the string

```
R 4.2.2 · ~/
> yes <- c("5'7\"", "6'2\"",  "5'12\"")
> no <- c("6,2\"", "6.2\"","I am 5'11\"", "3'2\"", "64")
> str_detect(yes, pattern)
[1] TRUE TRUE TRUE
> str_detect(no, pattern)
[1] FALSE FALSE FALSE FALSE FALSE
```

Working on some cases already. For now, we are permitting the inches to be 12 or larger. We will add a restriction later

# Recall \\s ?

```
R 4.2.2 · ~/
> paste(problems, collapse="  ")
[1] "6  5' 4\"  5.3  165cm  511  6  2  5'7  >9000  5'7\"  5'3\"  5 feet and
8.11 inches  5.25  5'11  5.5  11111  5'9''  6  6.5  150  5'10''  103.2  5.8
19  5  5.6  175  177  300  5,3  6'  6  5.9  6,8  5' 10  5.5  178  163  6.2  1
75  Five foot eight inches  6.2  5.8  5.1  178  165  5.11  5'5\"  165  180
5'2\"  5.75  169  5,4  7  5.4  157  6.1  169  5'3  5.6  214  183  5.6  6  162
178  180  5'10''  170  5'3''  178  0.7  190  5.4  184  5'7''  5.9  5'12  5.6
5.6  184  6  167  2'33  5'11  5'3\"  5.5  5.2  180  5.5  5.5  6.5  5,8  180
183  170  5'6''  172  612  5.11  168  5'4  1,70  172  87  5.5  176  5'7.5''
5'7.5''  111  5'2\"  173  174  176  175  5' 7.78\"  6.7  12  6  5.1  5.6  5.5
yyy  5.2  5'5  5'8  5'6  5 feet 7inches  89  5.6  5.7  183  172  34  25  6
5.9  168  6.5  170  175  6  22  5.11  684  6  1  1  6*12  5 .11  87  162  165
184  6  173  1.6  172  170  5.7  5.5  174  170  160  120  120  23  192  5 11
167  150  1.7  174  5.8  6  5'4  5'8\"  5'5  5.8  5.1  5.11  5.7  5'7  5'6
5'11\"  5'7\"  5'7  172  5'8  180  5' 11\"  5  180  180  6'1\"  5.9  5.2  5.5
69\"  5' 7\"  5'10''  5.51  5'10  5'10  5ft 9 inches  5 ft 9 inches  5'2  5'1
1  5.8  [illegible]  167  168  6  6.1  5'11''  5.69  178  182  164  5'8\"  185  6  86
5.7  70  [illegible]  5.25  5.5  5 feet 6 inches  5'10''  172  6  5'8  160  6'3\"  64
9,606  [illegible]  .1  152  1  180  728,346  175  158  173  164  6 04  169  0  18
5  168  5'[illegible]  169  5'5''  174  6.3  179  5'7\"  5.5  6  6  170  6  172  158  1
00  159  190  5.7  170  158  6'4\"  180  5.57  5'4  210  88  6  162  170 cm
5.7  170  157  186  170  7,283,465  5  5  34  161  5'6  5'6"
```

Some put a space between feet and inches, such as "5' 7\""

```
R 4.2.2 · ~/
> pattern_2 <- "^[4-7]'\\s\\d{1,2}\"$"
> str_subset(problems, pattern_2)
[1] "5' 4\""   "5' 11\"" "5' 7\""
```

Adding a \\s allows a space, but this will not match the patterns with no space. So do we need more than one regex pattern? It turns out we can use a quantifier for this as well

# Quantifier *, ?, +

```
R 4.2.2 · ~/
> yes <- c("AB", "A1B", "A11B", "A111B", "A1111B")
> data.frame(string = c("AB", "A1B", "A11B", "A111B", "A1111B"),
+            none_or_more = str_detect(yes, "A1*B"),
+            nore_or_once = str_detect(yes, "A1?B"),
+            once_or_more = str_detect(yes, "A1+B"))
  string none_or_more nore_or_once once_or_more
1     AB         TRUE         TRUE        FALSE
2    A1B         TRUE         TRUE         TRUE
3   A11B         TRUE        FALSE         TRUE
4  A111B         TRUE        FALSE         TRUE
5 A1111B         TRUE        FALSE         TRUE
```

*: match none or more instances

?: match none or once

+: match once or more instances

Be reminded to use * cautiously!

# Not ^

```
R 4.2.2 · ~/
> pattern <- "[^a-zA-Z]\\d"
> yes <- c(".3", "+2", "-0","*4")
> no <- c("A3", "B2", "C0", "E4")
> str_detect(yes, pattern)
[1] TRUE TRUE TRUE TRUE
> str_detect(no, pattern)
[1] FALSE FALSE FALSE FALSE
```

To specify patterns that we do not want to detect, we can use the ^ symbol with character class [].

Please distinguish the ^ outside [] that means the start of the string, and the ^ inside [] that means match anything but not the characters in [].

On the left is an examples if we want to detect digits that are preceded by anything except a letter.

Also recall

\\D: anything but not digits

\\W: anything but not alphanumerical

\\S: anything but not spaces

- 

```
R 4.2.2 · ~/
> yes <- c("1", "5", "9", "12")
> no <- c("123", "a4", "b")
> pattern <- "^.$"
> str_view_all(c(yes, no), pattern)
[1] | <1>
[2] | <5>
[3] | <9>
[4] | 12
[5] | 123
[6] | a4
[7] | <b>
> pattern <- "^..$"
> str_view_all(c(yes, no), pattern)
[1] | 1
[2] | 5
[3] | 9
[4] | <12>
[5] | 123
[6] | <a4>
[7] | b
> pattern <- "^...$"
> str_view_all(c(yes, no), pattern)
[1] | 1
[2] | 5
[3] | 9
[4] | 12
[5] | <123>
[6] | a4
[7] | b
```

. means matching any single character

.. means matching any single two characters

...

.* means matching whatever

```
R 4.2.2 · ~/
> pattern <- "^.*$"
> str_view_all(c(yes, no), pattern)
[1] | <1>
[2] | <5>
[3] | <9>
[4] | <12>
[5] | <123>
[6] | <a4>
[7] | <b>
```

# lookaround

Understand it as, do text matching but do not consume any text

Be careful, I teach lookaround only because it is a part of regex, avoid using them for good reasons! I will explain why later

```
R 4.2.2 · ~/
> pattern <- "(?=^\\w{8,16}$)(?=^[a-z|A-Z].*)(?=.*\\d+.*).*"
> yes <- c("Ihatepasswords1", "password1234")
> no <- c("sh0rt", "Ihaterpasswords", "7X%9,N`yrYG92b7")
> str_detect(yes, pattern)
[1] TRUE TRUE
> str_detect(no, pattern)
[1] FALSE FALSE FALSE
```

lookaround provide a way to ask for one or more conditions to be satisfied **without moving the search forward or matching it**. There are four types of lookaround: lookahead (?=pattern), lookbehind (?<=pattern), negative lookahead (?!pattern), and negative lookbehind (?<!pattern). You can concatenate them to check for multiple conditions.

On the left shows an example that checks if a password satisfies the following three conditions:

1) 8-16 alphanumerical characters
2) starts with a letter
3) has at least one digit

# Just don’t lookaround

```
R 4.2.2 · ~/
> pattern1 <- "^\\w{8,16}$"
> pattern2 <- "^[a-z|A-Z]"
> pattern3 <- ".*\\d+.*"
> str_detect(yes, pattern1) &
+ str_detect(yes, pattern2) &
+ str_detect(yes, pattern3)
[1] TRUE TRUE
> str_detect(no, pattern1) &
+ str_detect(no, pattern2) &
+ str_detect(no, pattern3)
[1] FALSE FALSE FALSE
```

You can achieve the task of the previous example with three individual regular expressions without using lookaround

Don’t use lookaround because they are error-prone and run slowly
/\ Just remember this

\/ OK if you don’t understand
To understand why it runs slow, we need some basic backgrounds of theory of computation. Regex without lookaround can be implemented with DFA (deterministic finite automaton), while regex with lookaround requires $\epsilon$-NFA (non-deterministic finite automaton with epsilon move). $\epsilon$-NFA can be converted to DFA (so regular expression with lookaround is still regular) but with a much larger number of states and transitions

# Group ()

```
R 4.2.2 · ~/
> pattern_without_groups <- "^[4-7],\\d*$"
> pattern_with_groups <-  "^([4-7]),(\\d*)$"
> yes <- c("5,9", "5,11", "6,", "6,1")
> no <- c("5'9", ",", "2,8", "6.1.1")
> s <- c(yes, no)
> str_detect(s, pattern_without_groups)
[1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE
> yes <- c("5,9", "5,11", "6,", "6,1")
> no <- c("5'9", ",", "2,8", "6.1.1")
> s <- c(yes, no)
> str_detect(s, pattern_with_groups)
[1]  TRUE  TRUE  TRUE  TRUE FALSE FALSE FALSE FALSE
> str_match(s, pattern_with_groups)
     [,1]   [,2] [,3]
[1,] "5,9"  "5"  "9"
[2,] "5,11" "5"  "11"
[3,] "6,"   "6"  ""
[4,] "6,1"  "6"  "1"
[5,] NA     NA   NA
[6,] NA     NA   NA
[7,] NA     NA   NA
[8,] NA     NA   NA
> str_extract(s, pattern_with_groups)
[1] "5,9"  "5,11" "6,"   "6,1"  NA     NA     NA     NA
```

Column 1, str_extract result
Column 2, group 1
Column 3, group 2

- So far, we used str_view_all to view the matching parts and str_detect to return a vector of logicals to tell the whether the strings match the pattern or not, however, a more power usage of regex is to find then extract the matches
- Group () is a powerful aspect of regex that permits the extraction of values
- Say we want to identify heights written like 5.6 as 5'6
- To extract values from defined groups, please use str_match, not str_extract. str_extract extracts the match of the whole pattern, while str_match extracts both the match of the whole pattern and the groups

# Search and replace with regex

```
R 4.2.2 · ~/
> pattern <- "^[4-7]'\\d{1,2}\"$"
> sum(str_detect(problems, pattern))
[1] 14
```

Earlier we defined the object problems containing the strings that do not appear to be in inches. We can see that not too many of our problematic strings match the pattern

```
R 4.2.2 · ~/
> problems[!str_detect(problems, pattern)]
 [1] "6"                      "5' 4\""
 [3] "5.3"                    "165cm"
 [5] "511"                    "6"
 [7] "2"                      "5'7"
 [9] ">9000"                  "5 feet and 8.11 inches"
[11] "5.25"                   "5'11"
[13] "5.5"                    "11111"
[15] "5'9''"                  "6"
```

To see those could not be matched

```
R 4.2.2 · ~/
> str_subset(problems, "inches")
[1] "5 feet and 8.11 inches" "Five foot eight inches"
[3] "5 feet 7inches"         "5ft 9 inches"
[5] "5 ft 9 inches"          "5 feet 6 inches"
```

Some used words feet and inches

```
R 4.2.2 · ~/
> str_subset(problems, "''")
 [1] "5'9''"   "5'10''"  "5'10''"  "5'3''"   "5'7''"   "5'6''"
 [7] "5'7.5''" "5'7.5''" "5'10''"  "5'11''"  "5'10''"  "5'5''"
```

Some used two ' instead of a single "

# Search and replace with regex

```
R 4.2.2 · ~/
> problems |>
+ str_replace("'''|\"", "") |>
+ str_detect("^[4-7]'\\d{1,2}$") |>
+ sum()
[1] 48
```

To match all x'y'', x'y", and x'y, let's remove the trailing symbols first (if there is any) and simply not use a symbol in the matching pattern

```
R 4.2.2 · ~/
> problems |>
+ str_replace("feet|ft|foot", "'") |>
+ str_replace("inches|in|''|\"", "") |>
+ str_detect("^[4-7]'\\d{1,2}$") |>
+ sum()
[1] 48
```

Similarly, we can replace feet|ft|foot with ', and remove inches|in, but the number of matches has not increased, what's the problem?

```
R 4.2.2 · ~/
> "5 feet 6 inches" |>
+ str_replace("feet|ft|foot", "'") |>
+ str_replace("inches|in|''|\"", "")
[1] "5 ' 6 "
```

There are spaces left in between the numbers and symbols, some might think of replacing " inches ", some might think of removing all spaces with str_replace_all

What could possibly go wrong

```
R 4.2.2 · ~/
> problems |>
+ str_replace("feet|ft|foot", "'") |>
+ str_replace("inches|in|''|\"", "") |>
+ str_detect("^[4-7]\\s*'\\s*\\d{1,2}$") |>
+ sum()
[1] 53
```

A better option is to allow some spaces through the pattern

# Search and replace using group

The second large type of problematic entries were of the form x.y, x,y and x y. We want to change all these to our common format x'y. But we can't just do a search and replace because we would change values such as 70.5 into 70'5. Our strategy will therefore be to search for a very specific pattern that assures us feet and inches are being provided and then, for those that match, replace appropriately. To do that we leverage the power of group

Previous we used str_match to extract the matching parts, but how can we use the matching parts directly in str_replace?

**The regex special character for the i-th group is \\i. So \\1 is the value extracted from the first group, \\2 the value from the second and so on**

```
R 4.2.2 · ~/
> pattern_with_groups <-  "^([4-7]),(\\d*)$"
> yes <- c("5,9", "5,11", "6,", "6,1")
> no <- c("5'9", ",", "2,8", "6.1.1")
> s <- c(yes, no)
> str_replace(s, pattern_with_groups, "\\1'\\2")
[1] "5'9"   "5'11"  "6'"    "6'1"   "5'9"   ","     "2,8"   "6.1.1"
```

# Search and replace using group

R 4.2.2 · ~/

```
> pattern_with_groups <-"^([4-7])[,\\.\\s+](\\d*)$"
> str_subset(problems, pattern_with_groups)
 [1] "5.3"  "5.25" "5.5"  "6.5"  "5.8"  "5.6"  "5,3"  "5.9"  "6,8"
[10] "5.5"  "6.2"  "6.2"  "5.8"  "5.1"  "5.11" "5.75" "5,4"  "5.4"
[19] "6.1"  "5.6"  "5.6"  "5.4"  "5.9"  "5.6"  "5.6"  "5.5"  "5.2"
[28] "5.5"  "5.5"  "6.5"  "5,8"  "5.11" "5.5"  "6.7"  "5.1"  "5.6"
[37] "5.5"  "5.2"  "5.6"  "5.7"  "5.9"  "6.5"  "5.11" "5.7"  "5.5"
[46] "5 11" "5.8"  "5.8"  "5.1"  "5.11" "5.7"  "5.9"  "5.2"  "5.5"
[55] "5.51" "5.8"  "5.7"  "6.1"  "5.69" "5.7"  "5.25" "5.5"  "5.1"
[64] "6 04" "6.3"  "5.5"  "5.7"  "5.57" "5.7"
> str_subset(problems, pattern_with_groups) |>
+ str_replace(pattern_with_groups, "\\1'\\2")
 [1] "5'3"  "5'25" "5'5"  "6'5"  "5'8"  "5'6"  "5'3"  "5'9"  "6'8"
[10] "5'5"  "6'2"  "6'2"  "5'8"  "5'1"  "5'11" "5'75" "5'4"  "5'4"
[19] "6'1"  "5'6"  "5'6"  "5'4"  "5'9"  "5'6"  "5'6"  "5'5"  "5'2"
[28] "5'5"  "5'5"  "6'5"  "5'8"  "5'11" "5'5"  "6'7"  "5'1"  "5'6"
[37] "5'5"  "5'2"  "5'6"  "5'7"  "5'9"  "6'5"  "5'11" "5'7"  "5'5"
[46] "5'11" "5'8"  "5'8"  "5'1"  "5'11" "5'7"  "5'9"  "5'2"  "5'5"
[55] "5'51" "5'8"  "5'7"  "6'1"  "5'69" "5'7"  "5'25" "5'5"  "5'1"
[64] "6'04" "6'3"  "5'5"  "5'7"  "5'57" "5'7"
```

# Testing and improving

```
R 4.2.2 · ~/
> not_inches_or_cm <- function(x, smallest = 50, tallest = 84){
+     inches <- suppressWarnings(as.numeric(x))
+     ind <- !is.na(inches) &
+         ((inches >= smallest & inches <= tallest) |
+              (inches/2.54 >= smallest & inches/2.54 <= tallest))
+     !ind
+ }
> problems <- reported_heights |>
+ filter(not_inches_or_cm(height)) |>
+ pull(height)
> length(problems)
[1] 200
```

```
R 4.2.2 · ~/
> converted <- problems |>
+ str_replace("feet|foot|ft", "'") |>
+ str_replace("inches|in|''|\"", "") |>
+ str_replace("^([4-7])[,\\.\\s+](\\d*)$", "\\1'\\2")
>
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.61
```

Developing the right regex on the first try is often difficult. Trial and error is a common approach to finding the regex pattern that satisfies all desired conditions.

Here we will test our approach, search for further problems, and tweak our approach for possible improvements.

Remember the sanity check function? Let's see what proportion of these fit our pattern after the processing steps we developed

It shows that we have matched well over half of the strings

# Testing and improving

```
R 4.2.2 · ~/
> converted[!index]
 [1] "6"            "165cm"        "511"          "6"
 [5] "2"            ">9000"        "5 ' and 8.11 " "11111"
 [9] "6"            "103.2"        "19"           "5"
[13] "300"          "6'"           "6"            "Five ' eight "
[17] "7"            "214"          "6"            "0.7"
[21] "6"            "2'33"         "612"          "1,70"
[25] "87"           "5'7.5"        "5'7.5"        "111"
[29] "5' 7.78"      "12"           "6"            "yyy"
[33] "89"           "34"           "25"           "6"
[37] "6"            "22"           "684"          "6"
[41] "1"            "1"            "6*12"         "5 .11"
[45] "87"           "6"            "1.6"          "120"
[49] "120"          "23"           "1.7"          "6"
[53] "5"            "69"           "5' 9 "        "5 ' 9 "
[57] "6"            "6"            "86"           "708,661"
[61] "5 ' 6 "       "6"            "649,606"      "10000"
[65] "1"            "728,346"      "0"            "6"
[69] "6"            "6"            "100"          "88"
[73] "6"            "170 cm"       "7,283,465"    "5"
[77] "5"            "34"
```

Let’s examine the remaining cases

**Four major patterns**:

1. Some students measuring exactly 5 or 6 feet did not enter any inches, for example 6', and our pattern requires that inches be included
2. Some students measuring exactly 5 or 6 feet entered just that number
3. Some of the inches were entered with decimal points. For example 5'7.5''. Our pattern only looks for two digits
4. Some entries have spaces at the end or elsewhere, for example “5' 9 ”, “5 ' 9 ”

# Testing and improving

```
R 4.2.2 · ~/
> converted <- problems |>
+ str_replace("feet|foot|ft", "'") |>
+ str_replace("inches|in|''|\"", "") |>
+ str_replace("^([4-7])[,\\.\\s+](\\d*)$", "\\1'\\2") |>
+ str_replace("^([56])'?$", "\\1'0")
>
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.73
```

# Few more things: str_trim()

Some are low hanging fruit if you knew the right way
To handle 4. Some entries have spaces at the end or elsewhere, for example “5' 9 ”, “5 ' 9 ”

```
R 4.2.2 · ~/
> converted <- problems |>
+ str_replace("feet|foot|ft", "'") |>
+ str_replace("inches|in|''|\"", "") |>
+ str_replace("^([4-7])[,\\.\\s+](\\d*)$", "\\1'\\2") |>
+ str_replace("^([56])'?$", "\\1'0") |>
+ str_trim()
>
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.745
```

str_trim() removes the leading and trailing spaces

# Few more things: str_to_lower()

Regex is case sensitive

So when handing whatever natural language with regex, it's a good idea to convert every single letter to  lower case before processing them

```
R 4.2.2 · ~/
> s <- c("Five Feet Eight Inches")
> str_to_lower(s)
[1] "five feet eight inches"
```

# Few more things: convert words to numbers

```
R 4.2.2 · ~/
> library(english)
> words_to_numbers <- function(s){
+     s <- str_to_lower(s)
+     for(i in 0:11)
+         s <- str_replace_all(s, words(i), as.character(i))
+     s
+ }
```

```
R 4.2.2 · ~/
> converted <- problems |>
+ words_to_numbers() |>
+ str_replace("feet|foot|ft", "'") |>
+ str_replace("inches|in|''|\"", "") |>
+ str_replace("^([4-7])[,\\.\\s+](\\d*)$", "\\1'\\2") |>
+ str_replace("^([56])'?$", "\\1'0") |>
+ str_trim()
>
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(converted, pattern)
> mean(index)
[1] 0.75
```

# Second try to put things together

```
R 4.2.2 · ~/
> words_to_numbers <- function(s){
+     s <- str_to_lower(s)
+     for(i in 0:11)
+         s <- str_replace_all(s, words(i), as.character(i))
+     s
+ }
> convert_format <- function(s){
+     s |>
+     str_replace("feet|foot|ft", "'") |>
+     str_replace("inches|in|''|\"", "") |>
+     str_replace("^([4-7])[,\\.\\s+](\\d*)$", "\\1'\\2") |>
+     str_replace("^([56])'?$", "\\1'0") |>
+     str_trim()
+ }
```

We now put all of what we have learned together into a function that takes a string vector and tries to convert as many strings as possible to one format. We write functions that puts together what we have done above

```
R 4.2.2 · ~/
> converted <- problems |> words_to_numbers() |> convert_format()
> remaining_problems <- converted[not_inches_or_cm(converted)]
> pattern <- "^[4-7]\\s*'\\s*\\d{1,2}$"
> index <- str_detect(remaining_problems, pattern)
> remaining_problems[!index]
 [1] "165cm"       "511"         "2"           ">9000"       "5 ' and 8.11" "11111"
 [7] "103.2"       "19"          "300"         "7"           "214"          "0.7"
[13] "2'33"        "612"         "1,70"        "87"          "5'7.5"        "5'7.5"
[19] "111"         "5' 7.78"     "12"          "yyy"         "89"           "34"
[25] "25"          "22"          "684"         "1"           "1"            "6*12"
[31] "5 .11"       "87"          "1.6"         "120"         "120"          "23"
[37] "1.7"         "86"          "708,661"     "649,606"     "10000"        "1"
[43] "728,346"     "0"           "100"         "88"          "170 cm"       "7,283,465"
[49] "34"
```

That's pretty much it

# Hold on, foot'inch needs to be converted to inch

foot'inch

```
R 4.2.2 · ~/
> converted
  [1] "6'0"          "5' 4"         "5'3"
  [4] "165cm"        "511"          "6'0"
  [7] "2"            "5'7"          ">9000"
 [10] "5'7"          "5'3"          "5 ' and 8.11"
 [13] "5'25"         "5'11"         "5'5"
 [16] "11111"        "5'9"          "6'0"
 [19] "6'5"          "5'10"         "103.2"
 [22] "5'8"          "19"           "5'0"
 [25] "5'6"          "300"          "5'3"
 [28] "6'0"          "6'0"          "5'9"
 [31] "6'8"          "5' 10"        "5'5"
```

inch

```
R 4.2.2 · ~/
> reported_heights$height
  [1] "75"          "70"          "68"
  [4] "74"          "61"          "65"
  [7] "66"          "62"          "66"
 [10] "67"          "72"          "6"
 [13] "69"          "68"          "69"
 [16] "66"          "75"          "64"
 [19] "60"          "67"          "66"
 [22] "5' 4\""      "70"          "73"
 [25] "72"          "69"          "69"
 [28] "72"          "64"          "72"
 [31] "75"          "71"          "67"
 [34] "66"          "67"          "69"
 [37] "68"          "66.75"       "72"
 [40] "5.3"         "69"          "68"
 [43] "63"          "60"          "73"
 [46] "74"          "74"          "66"
 [49] "68"          "73"          "70"
```

```
R 4.2.2 · ~/
> s <- c("5'10", "6'1")
> tab <- data.frame(x = s)
> tab |>
+ separate(x, c("feet", "inches"), sep = "'") |>
+ mutate_at(c("feet", "inches"), as.numeric) |>
+ mutate(height = feet * 12 + inches)
  feet inches height
1    5     10     70
2    6      1     73
```

First try using separate we have learned in reshaping data

mutate_at means apply a function at certain columns

# Doing the conversion, the regex way using extract()

R 4.2.2 · ~/

```
> tab |>
+ extract(x, c("feet", "inches"), regex = "(\\d)'(\\d{1,2})") |>
+ mutate_at(c("feet", "inches"), as.numeric) |>
+ mutate(height = feet * 12 + inches)
```

Groups in regex give us more flexibility

Once you started using regex with groups, separate and other less flexible functions are commonly out of business

# Second try to put things together (continued)

```r
pattern <- "^([4-7])\\s*'\\s*(\\d{1,2})$"

smallest <- 50
tallest <- 84
new_heights <- reported_heights |>
  mutate(original = height,
         height = height |> words_to_numbers() |> convert_format()) |>
  extract(height, c("feet", "inches"),
          regex = pattern, remove = FALSE) |>
  mutate_at(c("height", "feet", "inches"), as.numeric) |>
  mutate(guess = 12 * feet + inches) |>
  mutate(height = case_when(
    is.na(height) ~ as.numeric(NA), # allow non-convertibles
    between(height, smallest, tallest) ~ height, # use as is
    between(height/2.54, smallest, tallest) ~ height/2.54, # cm
    TRUE ~ as.numeric(NA))) |> # remainders to NA
  mutate(height = ifelse(is.na(height) & inches < 12 &
                           between(guess, smallest, tallest),
                         guess, height)) |>
  select(-guess) |>
  filter(!is.na(height))
```

R 4.2.2 · ~/

```
> str(new_heights)
'data.frame':   1039 obs. of  6 variables:
 $ time_stamp: chr  "2014-09-02 13:40:36" "2014-09-02 13:46:5
14-09-02 14:51:53" ...
 $ sex       : chr  "Male" "Male" "Male" "Male" ...
 $ height    : num  75 70 68 74 61 65 66 62 66 67 ...
 $ feet      : num  NA NA NA NA NA NA NA NA NA NA ...
 $ inches    : num  NA NA NA NA NA NA NA NA NA NA ...
 $ original  : chr  "75" "70" "68" "74" ...
> new_heights
             time_stamp    sex  height feet inches original
1   2014-09-02 13:40:36   Male 75.0000   NA     NA       75
2   2014-09-02 13:46:59   Male 70.0000   NA     NA       70
3   2014-09-02 13:59:20   Male 68.0000   NA     NA       68
4   2014-09-02 14:51:53   Male 74.0000   NA     NA       74
5   2014-09-02 15:16:15   Male 61.0000   NA     NA       61
6   2014-09-02 15:16:16 Female 65.0000   NA     NA       65
7   2014-09-02 15:16:19 Female 66.0000   NA     NA       66
8   2014-09-02 15:16:21 Female 62.0000   NA     NA       62
9   2014-09-02 15:16:21 Female 66.0000   NA     NA       66
10  2014-09-02 15:16:22   Male 67.0000   NA     NA       67
11  2014-09-02 15:16:22   Male 72.0000   NA     NA       72
12  2014-09-02 15:16:23   Male 72.0000    6      0        6
```

# String splitting (aka tokenization)

```
R 4.2.2 · ~/
> filename <- system.file("extdata/murders.csv", package = "dslabs")
> lines <- readLines(filename)
> lines |> head()
[1] "state,abb,region,population,total" "Alabama,AL,South,4779736,135"
[3] "Alaska,AK,West,710231,19"         "Arizona,AZ,West,6392017,232"
[5] "Arkansas,AR,South,2915918,93"     "California,CA,West,37253956,1257"
```

```
R 4.2.2 · ~/
> x <- str_split(lines, ",")
> x |> head()
[[1]]
[1] "state"      "abb"        "region"     "population" "total"

[[2]]
[1] "Alabama" "AL"      "South"   "4779736" "135"

[[3]]
[1] "Alaska" "AK"      "West"    "710231"  "19"
```

```
R 4.2.2 · ~/
> col_names <- x[[1]]
> x <- x[-1]
```

Another very common data wrangling operation is string splitting. To illustrate how this comes up, we start with an illustrative example. Suppose we did not have the function read_csv or read.csv available to us. We instead have to read a csv file using the base R function readLines

We want to extract the values that are separated by a comma for each string in the vector. The command str_split does exactly this

Note that the first entry has the column names, so we can separate that out

https://stackoverflow.com/questions/7336679/in-r-what-does-a-negative-index-do

# String splitting (aka tokenization)

R 4.2.2 · ~/

```
> x <- str_split(lines, ",", simplify = TRUE)
> col_names <- x[1,]
> x <- x[-1,]
> colnames(x) <- col_names
> x |> as_data_frame() |>
+ mutate_all(parse_guess)
# A tibble: 51 × 5
   state                abb   region    population total
   <chr>                <chr> <chr>          <dbl> <dbl>
 1 Alabama              AL    South        4779736   135
 2 Alaska               AK    West          710231    19
 3 Arizona              AZ    West         6392017   232
 4 Arkansas             AR    South        2915918    93
 5 California           CA    West        37253956  1257
 6 Colorado             CO    West         5029196    65
 7 Connecticut          CT    Northeast    3574097    97
 8 Delaware             DE    South         897934    38
 9 District of Columbia DC    South         601723    99
10 Florida              FL    South       19687653   669
# … with 41 more rows
```

A more convenient way:

Using the parse_guess function from readr with mutate_all

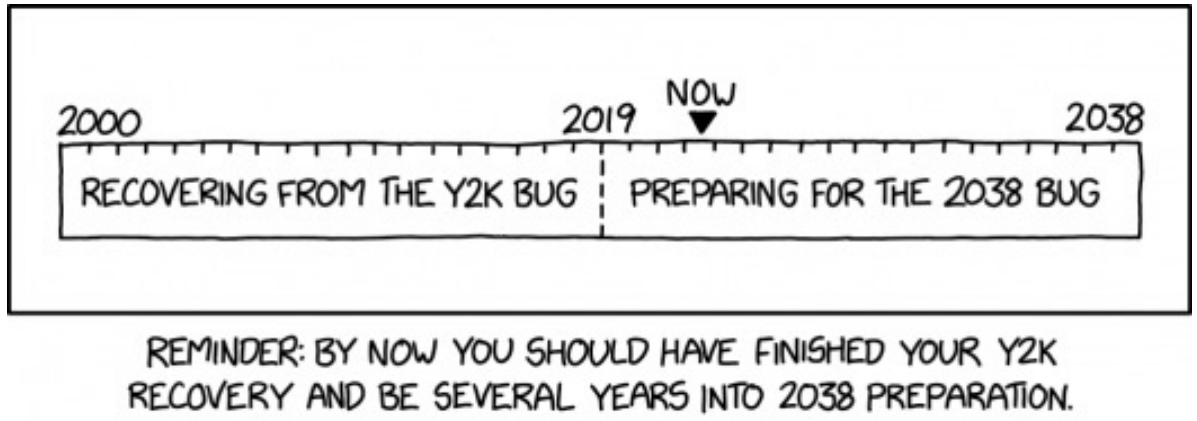
https://www.rdocumentation.org/packages/readr/versions/2.1.3/topics/parse_guess

# The epochalypse


The Unix epoch - number of seconds elapsed since *00:00:00 UTC on 1 January 1970*

This is stored as a signed 32-bit integer, thus maxed at $2^{31} - 1$

*03:14:07 UTC on 19 January 2038* will be the $(2^{31} - 1)$ second since the Unix epoch



# Parsing dates and times

We have described three main types of vectors: numeric, character, and logical. In data science projects, we very often encounter variables that are dates. Although we can represent a date with a string, for example November 2, 2017, once we pick a reference day, referred to as the epoch, they can be converted to numbers by calculating the number of days since the epoch. Computer languages usually use January 1, 1970, as the epoch. So, for example, January 2, 2017 is day 1, December 31, 1969 is day -1, and November 2, 2017, is day 17,204

Now how should we represent dates and times when analyzing data in R? We could just use days since the epoch, but then it is almost impossible to interpret. If I tell you it's November 2, 2017, you know what this means immediately. If I tell you it's day 17,204, you will be quite confused. Similar problems arise with times and even more complications can appear due to time zones

For this reason, R defines a data type just for dates and times

```
R 4.2.2 · ~/
> library(tidyverse)
> library(dslabs)
> data("polls_us_election_2016")
> polls_us_election_2016$startdate |> head()
[1] "2016-11-03" "2016-11-01" "2016-11-02" "2016-11-04"
[5] "2016-11-03" "2016-11-03"
> class(polls_us_election_2016$startdate)
[1] "Date"
```

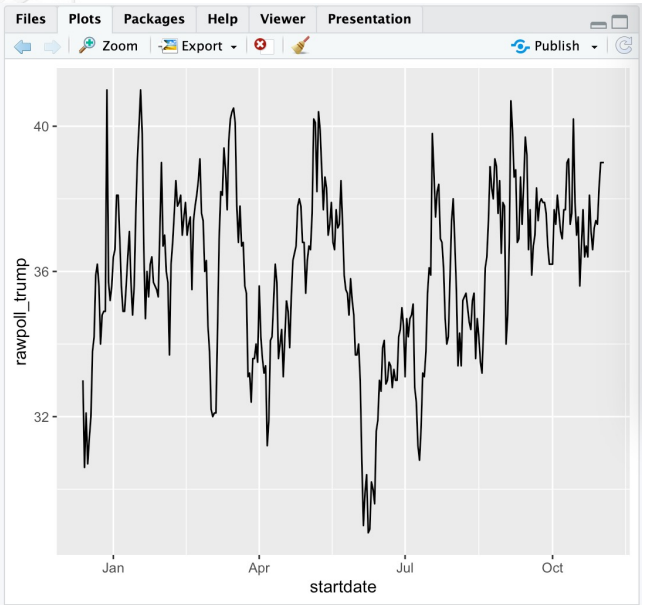
# Parsing dates and times

Look at what happens when we convert them to numbers:

```
R 4.2.2 · ~/
> as.numeric(polls_us_election_2016$startdate) |> head()
[1] 17108 17106 17107 17109 17108 17108
```

It turns them into days since the epoch. The as.Date function can convert a character into a date. So to see that the epoch is day 0 we can type

```
R 4.2.2 · ~/
> as.Date("1970-01-01") |> as.numeric()
[1] 0
```




Plotting functions, such as those in ggplot, are aware of the date format

```
R 4.2.2 · ~/
> polls_us_election_2016 |> filter(pollster == "Ipsos" & state =="U.S.") |>
+ ggplot(aes(startdate, rawpoll_trump)) +
+ geom_line()
```

# The lubridate package

```
R 4.2.2 · ~/
> library(lubridate)
```

The tidyverse includes functionality for dealing with dates through the lubridate package

```
R 4.2.2 · ~/
> set.seed(42)
> dates <- sample(polls_us_election_2016$startdate, 10) |> sort()
> dates
 [1] "2016-02-16" "2016-02-22" "2016-05-23" "2016-09-06" "2016-09-16"
 [6] "2016-09-27" "2016-10-18" "2016-10-25" "2016-11-01" "2016-11-01"
```

We will take a random sample of dates to show some of the useful things one can do

```
R 4.2.2 · ~/
> data.frame(date = dates,
+ month = month(dates),
+ day = day(dates),
+ year = year(dates))
```

| | date | month | day | year |
|---|---|---|---|---|
| 1 | 2016-02-16 | 2 | 16 | 2016 |
| 2 | 2016-02-22 | 2 | 22 | 2016 |
| 3 | 2016-05-23 | 5 | 23 | 2016 |
| 4 | 2016-09-06 | 9 | 6 | 2016 |
| 5 | 2016-09-16 | 9 | 16 | 2016 |
| 6 | 2016-09-27 | 9 | 27 | 2016 |
| 7 | 2016-10-18 | 10 | 18 | 2016 |
| 8 | 2016-10-25 | 10 | 25 | 2016 |
| 9 | 2016-11-01 | 11 | 1 | 2016 |
| 10 | 2016-11-01 | 11 | 1 | 2016 |

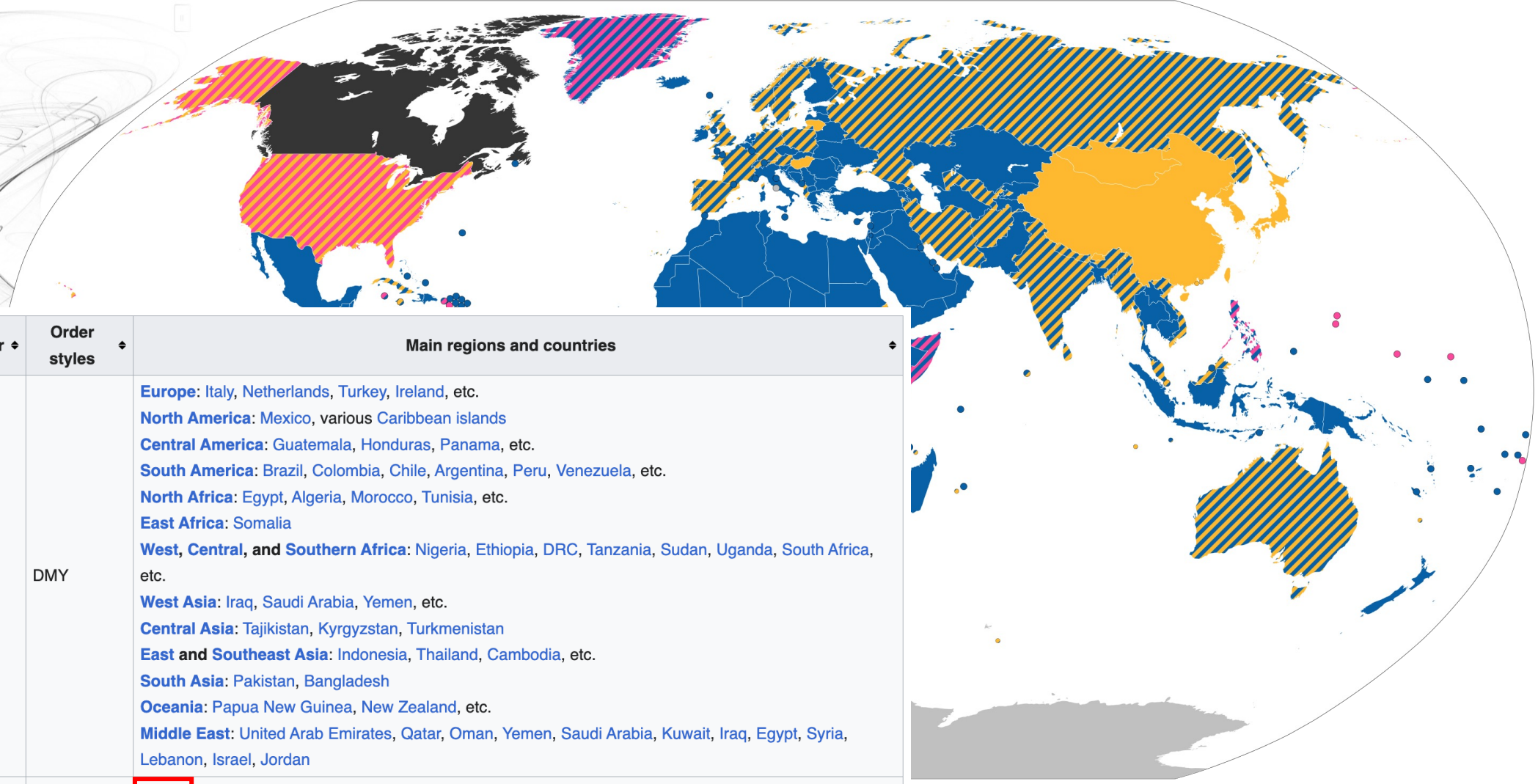
We will take a random sample of dates to show some of the useful things one can do

```
R 4.2.2 · ~/
> month(dates, label = TRUE)
 [1] Feb Feb May Sep Sep Sep Oct Oct Nov Nov
12 Levels: Jan < Feb < Mar < Apr < May < Jun < Jul < Aug < ... < Dec
```

We can also extract the month labels

```
R 4.2.2 · ~/
> x <- c(20090101, "2009-01-02", "2009 01 03", "2009-1-4",
+        "2009-1, 5", "Created on 2009 1 6", "200901 !!! 07")
> ymd(x)
[1] "2009-01-01" "2009-01-02" "2009-01-03" "2009-01-04" "2009-01-05"
[6] "2009-01-06" "2009-01-07"
```

Another useful set of functions are the parsers that convert strings into dates. The function ymd assumes the dates are in the format YYYY-MM-DD and tries to parse as well as possible



| Colour | Order styles | Main regions and countries |
|---|---|---|
| | DMY | **Europe**: Italy, Netherlands, Turkey, Ireland, etc.<br>**North America**: Mexico, various Caribbean islands<br>**Central America**: Guatemala, Honduras, Panama, etc.<br>**South America**: Brazil, Colombia, Chile, Argentina, Peru, Venezuela, etc.<br>**North Africa**: Egypt, Algeria, Morocco, Tunisia, etc.<br>**East Africa**: Somalia<br>**West, Central, and Southern Africa**: Nigeria, Ethiopia, DRC, Tanzania, Sudan, Uganda, South Africa, etc.<br>**West Asia**: Iraq, Saudi Arabia, Yemen, etc.<br>**Central Asia**: Tajikistan, Kyrgyzstan, Turkmenistan<br>**East and Southeast Asia**: Indonesia, Thailand, Cambodia, etc.<br>**South Asia**: Pakistan, Bangladesh<br>**Oceania**: Papua New Guinea, New Zealand, etc.<br>**Middle East**: United Arab Emirates, Qatar, Oman, Yemen, Saudi Arabia, Kuwait, Iraq, Egypt, Syria, Lebanon, Israel, Jordan |
| | YMD | China, Japan, South Korea, Taiwan, Hungary, Mongolia, Lithuania, Bhutan |
| | MDY | Some US island territories |
| | DMY, YMD | India, Russia, Vietnam, Germany, Iran, United Kingdom, France, Myanmar, Spain, Poland, Uzbekistan, Afghanistan, Nepal, Australia, Cameroon, Sri Lanka, Malaysia, Singapore, etc. |
| | DMY, MDY | Philippines, Togo, Puerto Rico, Cayman Islands, Greenland |
| | MDY, YMD | United States |
| | MDY, DMY, YMD | Kenya, Canada, Ghana |

# The lubridate package

```
R 4.2.2 · ~/
> x <- "09/01/02"
> dmy(x)
[1] "2002-01-09"
> mdy(x)
[1] "2002-09-01"
```

To deal with dd mm yyyy, use dmy

To deal with mm dd yyyy, use mdy

```
R 4.2.2 · ~/
> now()
[1] "2023-03-15 13:24:31 HKT"
> now("GMT")
[1] "2023-03-15 05:24:37 GMT"
```

The lubridate package is also useful for dealing with times. In R base, you can get the current time typing Sys.time(). The lubridate package provides a slightly more advanced function, now, that permits you to define the time zone

```
R 4.2.2 · ~/
> now() |> hour()
[1] 13
> now() |> minute()
[1] 26
> now() |> second()
[1] 53.60153
```

We can also extract hours, minutes, and seconds

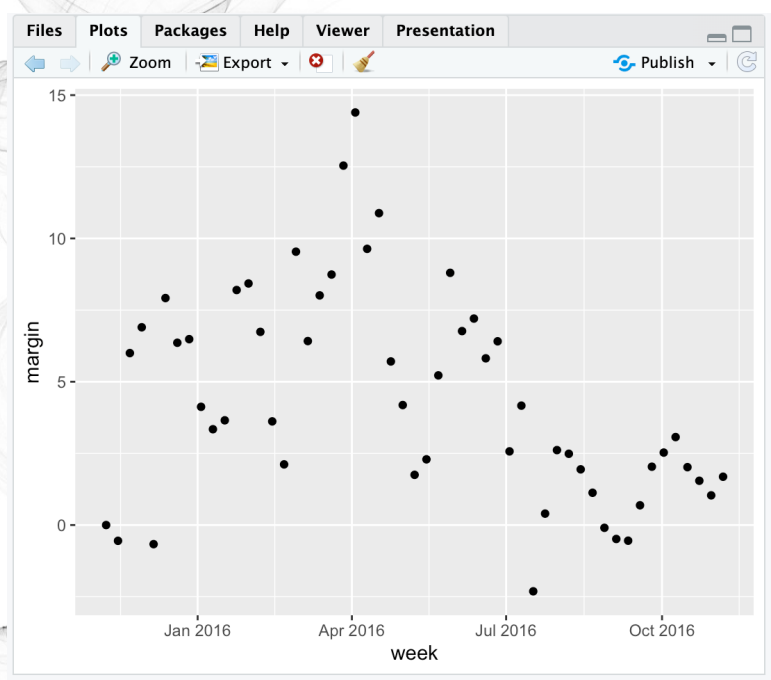
# More useful lubridate package functions

```
R 4.2.2 · ~/
> x <- c("12:34:56")
> hms(x)
[1] "12H 34M 56S"
> x <- "Nov/2/2012 12:34:56"
> mdy_hms(x)
[1] "2012-11-02 12:34:56 UTC"
```

hms, mdy_hms

```
R 4.2.2 · ~/
> make_date(2019, 7, 6)
[1] "2019-07-06"
```

make_date




round_date can round dates to nearest year, quarter, month, week, day, hour, minutes, or seconds. So if we want to group all the polls by week of the year we can do the following:

```
R 4.2.2 · ~/
> polls_us_election_2016 |>
+ mutate(week = round_date(startdate, "week")) |>
+ group_by(week) |>
+ summarize(margin = mean(rawpoll_clinton - rawpoll_trump)) |>
+ ggplot(aes(week, margin)) +
+ geom_point()
```